

Unit-I

Q1

Explain the MVC (Model-View-Controller) architecture. Describe the role of each component and illustrate with a real-world example such as user login

Q2

Compare Monolithic and Microservices architectures. Discuss their characteristics, advantages, disadvantages, and suitable use cases with examples.

Q3

Explain HTTP Request and HTTP Response messages with their components, status codes, and examples.

Q4

Differentiate between client-side and server-side technologies. Give examples of languages, execution environments, security, and performance for each.

Q5

Compare Apache, Nginx, and IIS web servers. Discuss their features, performance differences, configuration, OS support, and use cases.

Unit-II

Q1

Explain the various data types in Python with examples. Include Numbers, String, List, Tuple, and Dictionary.

Q2

Explain the different types of operators in Python with examples Arithmetic, Relational, Assignment, Logical, Bitwise, Membership, and Identity.

Q3

Discuss the need and importance of functions in Python. Explain function definition, function call, flow of execution, and the four function prototypes with example programs.

Q4

Discuss the various types of arguments in Python Required, Keyword, Default, and Variable Length with example programs for each.

Q5

Explain the operations that can be performed on List and Tuple in Python (minimum 5 operations each) with example programs. Also differentiate List vs Tuple.

Q6

Write Python programs for: (i) Swapping two variables, (ii) Finding distance between two points, (iii) Circulating values of n variables.

Q7

Develop a Python program to demonstrate File Handling operations using Text, CSV, and JSON files. The program should:

1. Create and write student records into a text file.
2. Read and display the contents of the text file.
3. Store student details (Name, Roll, Marks) into a CSV file using the `csv` module.
4. Convert the student data into JSON format and save it into a JSON file using the `json` module.
5. Read the JSON file and display the stored records.
6. Use the `with` statement for proper file handling and explain the importance of different file modes (`r`, `w`, `a`, `r+`).

Unit-III

1. Explain Flask Framework. Discuss its features, advantages, and application structure.
2. Explain the process of setting up a Flask environment and creating a basic Flask application with an example.
3. What are Routes, Views, and Templates in Flask? Explain with suitable examples.
4. Explain HTTP Request and Response Handling in Flask.
5. Explain Form Submission and Input Validation in Flask.
6. What are Cookies and Sessions in Flask? Explain their working with examples.
7. Explain Flask Redirect, Message Flashing, and File Uploading with examples.
8. Explain Sending Email in Flask using Flask-Mail.

Q9

Develop a Flask Web Application for Student Registration System with Form Handling and Session Management.

The application should include the following features:

1. Create a Flask application with routes for:
 - Home Page
 - About Page
 - Student Registration Form
2. Implement HTML templates using Jinja2.
3. Accept student details using an HTML form:
 - Name
 - Email
 - Course
4. Handle form submission using HTTP POST method.
5. Validate user input and display submitted data on another page.
6. Use Sessions or Cookies to store the student username.
7. Implement Flask redirect and flash messages for successful form submission.

8. Add a simple file upload feature for uploading student documents/photos.

Q10

Develop a Flask-based Student Login and File Upload System using Routes, Templates, Sessions, and Form Handling.

The application should perform the following tasks:

1. Create a Flask application with:
 - Home Page
 - Login Page
 - Dashboard Page
2. Implement routing using `@app.route()`.
3. Create HTML templates inside the `templates` folder.
4. Accept username and password using an HTML form.
5. Handle GET and POST requests using Flask request methods.
6. Validate login credentials and:
 - Display flash message for successful/failed login.
 - Redirect user to dashboard after successful login.
7. Use Sessions or Cookies to store logged-in user information.
8. Implement file upload functionality to upload a profile image/document.
9. Explain the project structure of Flask (`app.py`, `templates`, `static`).
